



Samueli
School of Engineering

CS-M151B

Computer Systems Architecture

Blaise Tine
UCLA Computer Science

Agenda

- Midterm grading
- Project #2

SuperScalar Processor

- Scalar: can process at most one instruction per cycle
- SuperScalar: can process at most N instructions per cycle

Out of Order Execution

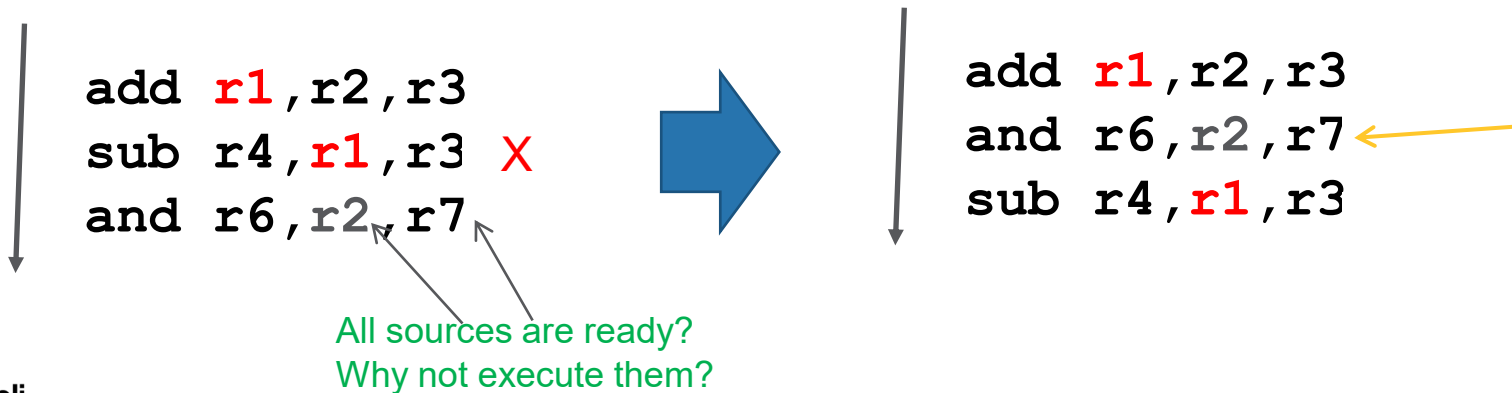
- If the next instruction cannot move forwards due to dependencies, **we skip it and pick the next one.**
- The **instruction window** size determines pipeline utilization

Out of Order Execution

- OoO Design Challenges?
 - Finding a ready instruction
 - How do we track dependencies when ordering is broken?
 - How do we handle pipeline hazards?
 - How do we preserve ordering for external states?
 - Registers and memory

Static Scheduling

- Compiler re-order the instruction
 - Internally track dependencies
 - Check register use
- How do we do that in the compiler?



Dynamic Scheduling

- How to reduce register dependencies?

Dynamic Scheduling

- Can we increase the number of registers to work with?
 - You cannot change the ISA (our S/W contract)

Dynamic Scheduling

- **Solution:**
 - Use internal registers: “physical” registers
 - We will call ISA registers “logical” or “architected”
 - initially map logical registers to physical registers

HW Register Renaming

Needed Features:

- mapping mechanism
- physical registers
 - allocation/deallocation mechanism
 - allocated vs. free registers

HW Register Renaming

Exercise:

- Identify all data dependencies

Program code

I1: ADD R1, R3, R3

I2: SUB R2, R1, R5

I3: AND R5, R11, R7

I4: OR R8, R5, R6

I5: XOR R2, R4, R11

RAW 
WAR 
WAW 

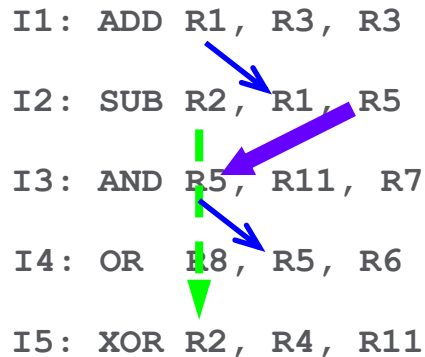
HW Register Renaming

Exercise

- 2 RAWs
- 1 WAR
- 1 WAW
- Which instructions cannot be reordered?

Program code

```
I1: ADD R1, R3, R3
I2: SUB R2, R1, R5
I3: AND R5, R11, R7
I4: OR R8, R5, R6
I5: XOR R2, R4, R11
```



RAW →
WAR →
WAW - - →

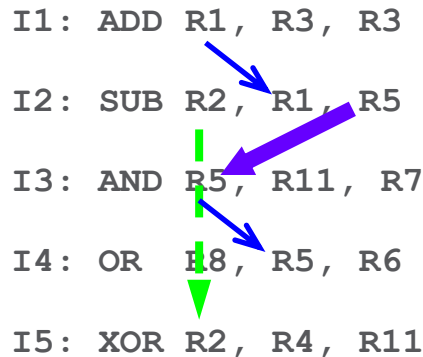
HW Register Renaming

Exercise: can you reorder instructions?

- I2 can not exec before I1 because I2 need I1 result
- I4 can not exec before I3 because I4 need I3 result
- I3 can not exec before I2 because I3 will overwrite R5
- I5 can not go before I2 because it will overwrite R2 with a stale value
- What about I5 before I3?

Program code

```
I1: ADD R1, R3, R3
I2: SUB R2, R1, R5
I3: AND R5, R11, R7
I4: OR R8, R5, R6
I5: XOR R2, R4, R11
```



RAW →
WAR →
WAW →

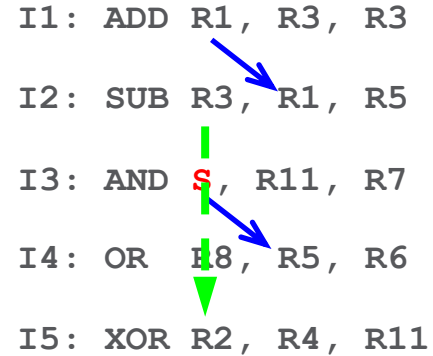
HW Register Renaming

Example

- Let's rename R5 to decouple I2 and I3
- R5 = S
- Removed WAR

Program code

```
I1: ADD R1, R3, R3
I2: SUB R3, R1, R5
I3: AND S, R11, R7
I4: OR R8, R5, R6
I5: XOR R2, R4, R11
```



RAW →

WAR →

WAW - - →

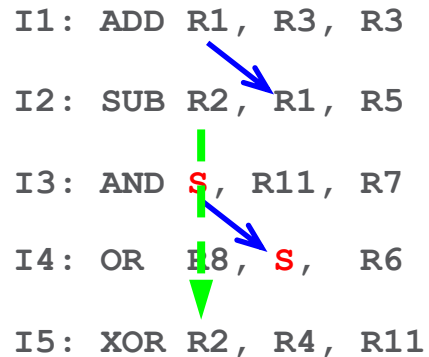
HW Register Renaming

Example

- I4 also needs to change since R5 is S

Program code

```
I1: ADD R1, R3, R3
I2: SUB R2, R1, R5
I3: AND S, R11, R7
I4: OR R8, S, R6
I5: XOR R2, R4, R11
```



RAW 
WAR 
WAW 

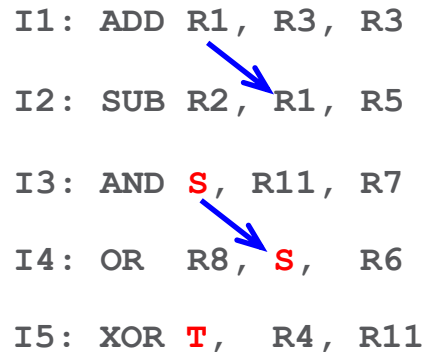
HW Register Renaming

Example

- Let's rename R2 to decouple I2 and I5
- R2 = T
- Removed WAW

Program code

```
I1: ADD R1, R3, R3
I2: SUB R2, R1, R5
I3: AND S, R11, R7
I4: OR R8, S, R6
I5: XOR T, R4, R11
```



RAW 
WAR 
WAW 

HW Register Renaming

Example

- Let's rename R1 to r
- Did we remove RAW?

Program code

I1: ADD r, R3, R3

I2: SUB R2, r, R5

I3: AND S, R11, R7

I4: OR R8, S, R6

I5: XOR T, R4, R11

RAW →
WAR →
WAW - - →

HW Register Renaming

Register renaming eliminates

- WAR dependencies
- WAW dependencies
- **We still have RAW dependencies**

Program code

I1: ADD **r**, R3, R3

I2: SUB R2, **r**, R5

I3: AND **s**, R11, R7

I4: OR R8, **s**, R6

I5: XOR **t**, R4, R11

RAW 
WAR 
WAW 

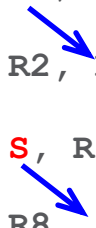
HW Register Renaming

Implementation

- Space for **S**, **T**, etc.
- How do we know when to rename a register?

Program code

```
I1: ADD R1, R3, R3
I2: SUB R2, R1, R5
I3: AND S, R11, R7
I4: OR R8, S, R6
I5: XOR T, R4, R11
```



RAW 
WAR 
WAW 

HW Register Renaming

Simple Solution

- Do renaming for every instruction
- Change the name of a register each time we decode an instruction that will write to it.
- Remember what name we gave it 😊

Program code

I1: ADD R1, R3, R3

I2: SUB R2, R1, R5

I3: AND S, R11, R7

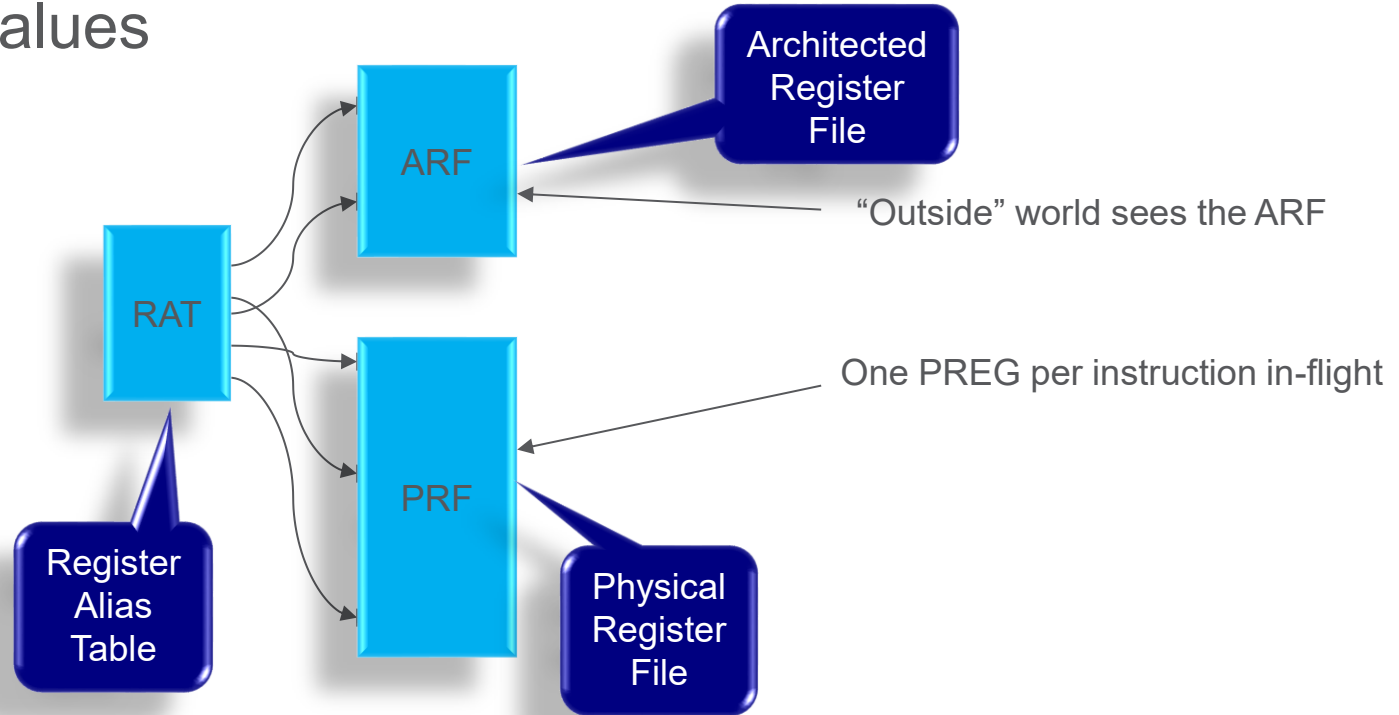
I4: OR R8, S, R6

I5: XOR T, R4, R11

RAW →
WAR →
WAW - - →

Register File Organization

We need some physical structure to store the register values

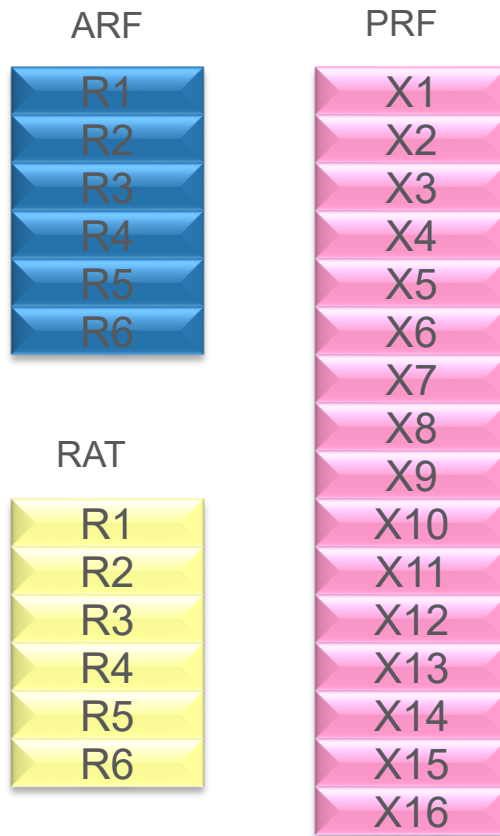


Register File Organization

- RAT stores ARF to PRF mappings
- The RAT is initialized to Identity

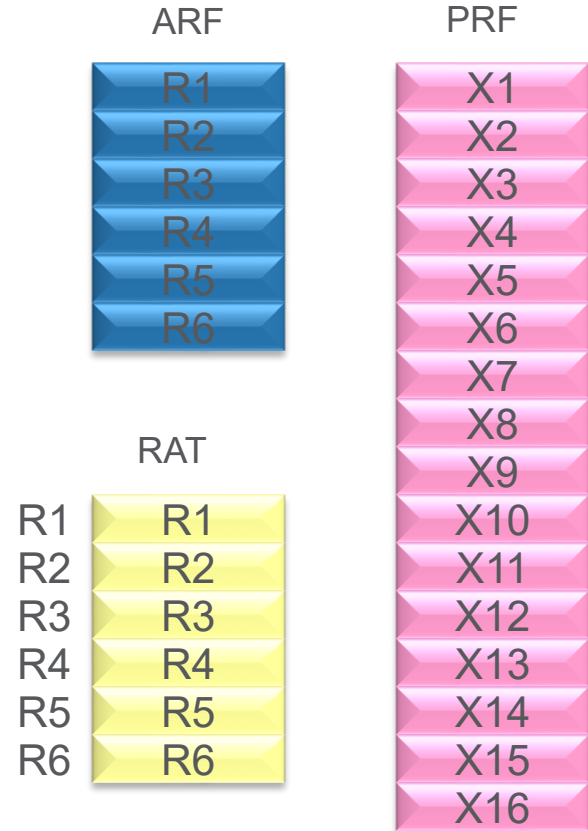
3-Step Renaming Process:

- Use the RAT to replace src registers
 - $src_p := RAT[src];$
- Assign a new PRF to each dst register
 - $dst_p := PRF.pop();$
- Update the RAT:
 - $RAT[dst] := dst_p;$



Register File Organization

- How would you implement the PRF?
- How would you implement the RAT?
- At which stage should we do it?



Whiteboard: Renaming Implementation

Whiteboard: RAW

$R1 = R2 + R3$

$R2 = R4 - R1$

$R1 = R3 * R6$

$R2 = R1 + R2$

$R3 = R1 \gg 1$

BNEZ R3, top

Whiteboard: WAR

$R1 = R2 + R3$

$R2 = R4 - R1$

$R1 = R3 * R6$

$R2 = R1 + R2$

$R3 = R1 \gg 1$

BNEZ R3, top

Whiteboard: WAW

$R1 = R2 + R3$

$R2 = R4 - R1$

$R1 = R3 * R6$

$R2 = R1 + R2$

$R3 = R1 \gg 1$

BNEZ R3, top

Whiteboard: Renaming

R1 = R2 + R3

R2 = R4 - R1

R1 = R3 * R6

R2 = R1 + R2

R3 = R1 >> 1

BNEZ R3, top

RAT	
R1	R1
R2	R2
R3	R3
R4	R4
R5	R5
R6	R6

Free pool: X9, X11, X7, X2, X13, X4, X8, X12, X3, X5...

Putting it all together

top:

$R1 = R2 + R3$

$R2 = R4 - R1$

$R1 = R3 * R6$

$R2 = R1 + R2$

$R3 = R1 \gg 1$

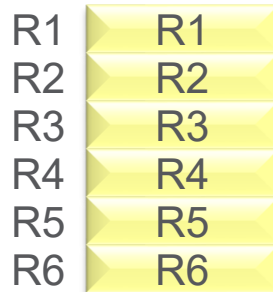
BNEZ R3, top

Free pool:

X9, X11, X7, X2,
X13, X4, X8,
X12, X3, X5...



RAT



1. List all instructions for 2 loop iterations
2. Show graph when only RAW enforced
3. Show graph when all deps enforced
4. Rename the registers

Putting it all together

R1 = R2 + R3
R2 = R4 - R1
R1 = R3 * R6
R2 = R1 + R2
R3 = R1 >> 1
BNEZ R3, top
R1 = R2 + R3
R2 = R4 - R1
R1 = R3 * R6
R2 = R1 + R2
R3 = R1 >> 1
BNEZ R3, top

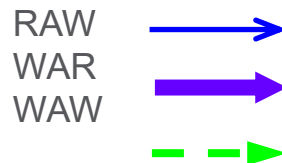
2. Show graph when only RAW enforced



Putting it all together

R1 = R2 + R3
R2 = R4 - R1
R1 = R3 * R6
R2 = R1 + R2
R3 = R1 >> 1
BNEZ R3, top
R1 = R2 + R3
R2 = R4 - R1
R1 = R3 * R6
R2 = R1 + R2
R3 = R1 >> 1
BNEZ R3, top

3. Show graph when all deps enforced



Putting it all together

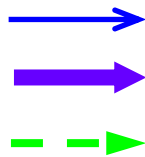
X9 = R2 + R3
X11 = R4 - X9
X7 = R3 * R6
X2 = X7 + X11
X13 = X7 >> 1
BNEZ X13, top
X14 = X2 + X13
X8 = R4 - X14
X12 = X13 * R6
X3 = X12 + X8
X5 = X12 >> 1
BNEZ X5, top

4. Rename the registers

Free pool:

~~X9, X11~~, X7, X2,
X13, X4, X8,
X12, X3, X5...

RAW
WAR
WAW



Putting it all together

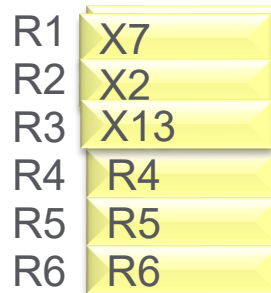
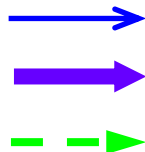
X9 = R2 + R3
X11 = R4 - X9
X7 = R3 * R6
X2 = X7 + X11
X13 = X7 >> 1
BNEZ X13, top
X14 = X2 + X13
X8 = R4 - X14
X12 = X13 * R6
X3 = X12 + X8
X5 = X12 >> 1
BNEZ X5, top

4. Rename the registers

Free pool:

~~X9, X11, X7, X2,~~
X13, X4, X8,
X12, X3, X5...

RAW
WAR
WAW



HW Register Renaming

We keep using new physical registers

- What happens when we run out?
- When do we recycle physical registers?

HW Register Renaming

We keep using new physical registers

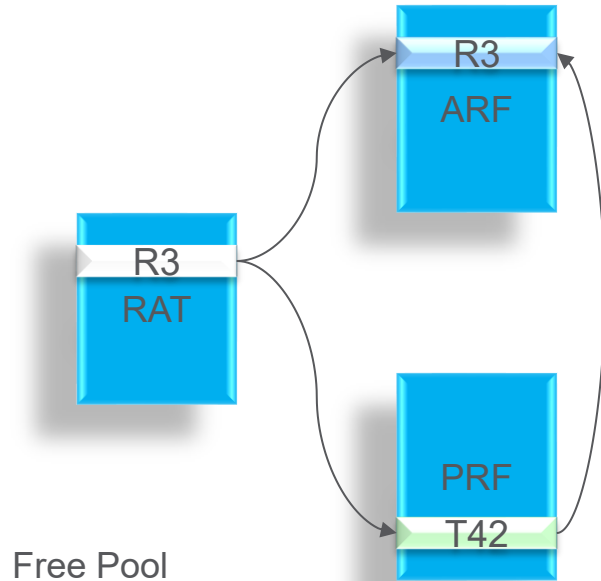
- What happens when we run out?
- We stall the pipeline until they are freed

HW Register Renaming

When do we recycle physical registers?

- When not needed anymore
 - Last instruction using it has executed.
- Need a tracking mechanism

HW Register Renaming



Architected register file contains the “official” processor state

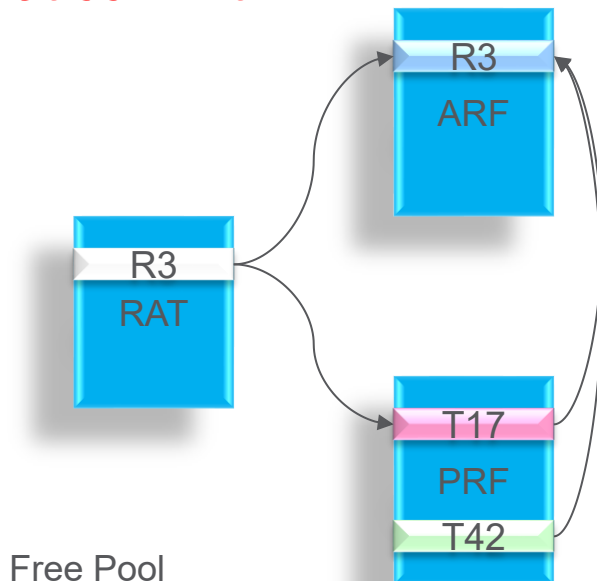
When an instruction commits, it makes its result “official” by updating the ARF

The ARF now contains the correct value; update the RAT

T42 is no longer needed, return to the physical register free pool

HW Register Renaming

R3 could be using a
different PRF at commit!



Update ARF as usual

Deallocate physical register

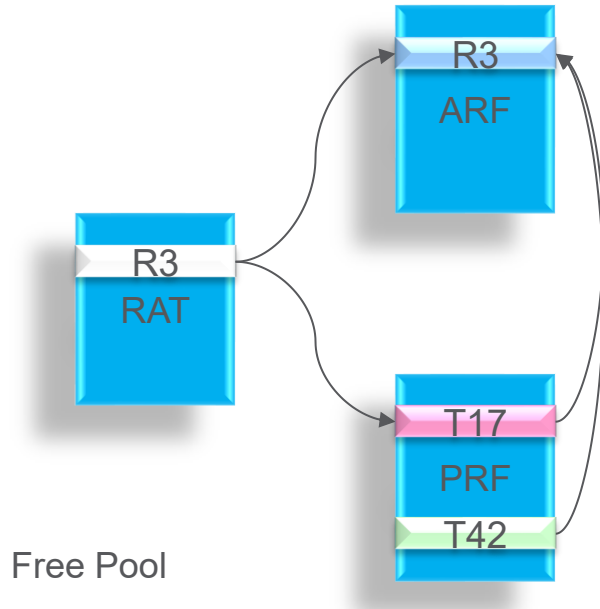
Don't touch the RAT!
(Someone else is the most
recent writer to R3)

At some point in the future,
the newer writer of R3 exits

This instruction was the most
recent writer, *now* update the RAT

Deallocate physical register

HW Register Renaming



How does the CPU know which ARF to update?

The RAT only track ARF-> PRF mapping

Implementations typically integrate PRF->ARF mapping into another structure called the Reorder Buffer (ROB)

Whiteboard: RAT Update

Attendance

- Please Fill the form with today's keyword to register your attendance.



Processor Types (Recap)

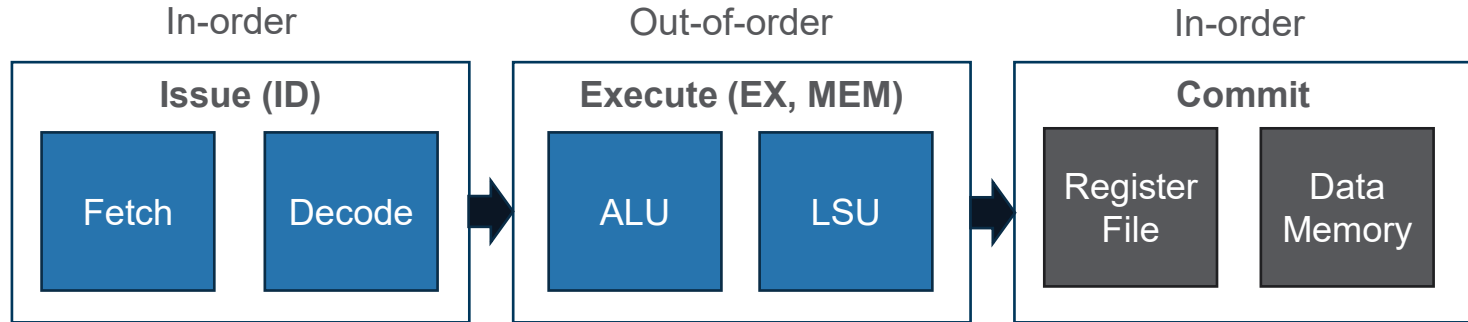
- Scalar CPU: process at most one instruction per cycle
 - Shortcomings: $IPC \leq 1$
 - Why?
- SuperScalar CPU : process at most N instructions per cycle
 - Shortcomings: IPC rarely reach N
 - Why?
- Out of Order CPU: process instructions out of order
 - Goal: Resolve data dependencies

Modern Processor Requirements

- Fetching multiple instructions each cycle
- Executing multiple instructions concurrently
- Partitioned execute stage
 - Multiple functional units (e.g. ALU, FPU, LSU)
 - Variable-latency units
- Commit instructions in order
 - Preserving external states

Modern Processor Pipeline

- Three main stages: Issue, Execute, Commit

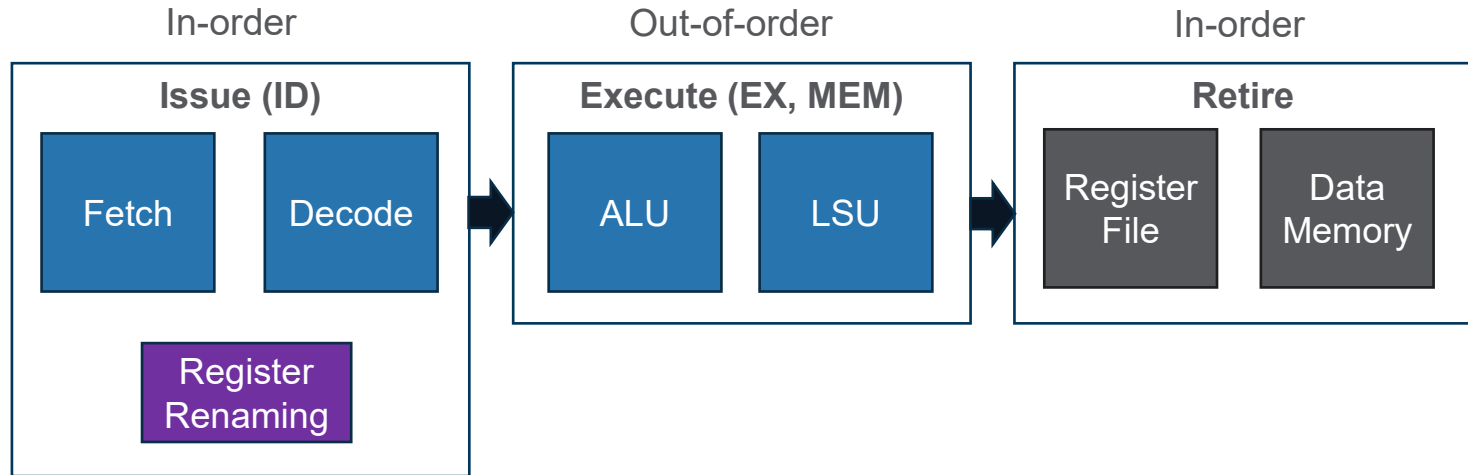


Out of Order Execution

- OoO Design Challenges?
 - How do we resolve data dependencies?
 - How do we handle pipeline hazards?
 - How do we preserve ordering for external states?

Register Renaming

- Resolve WAR and WAW dependencies
- Reduces pipeline stalls

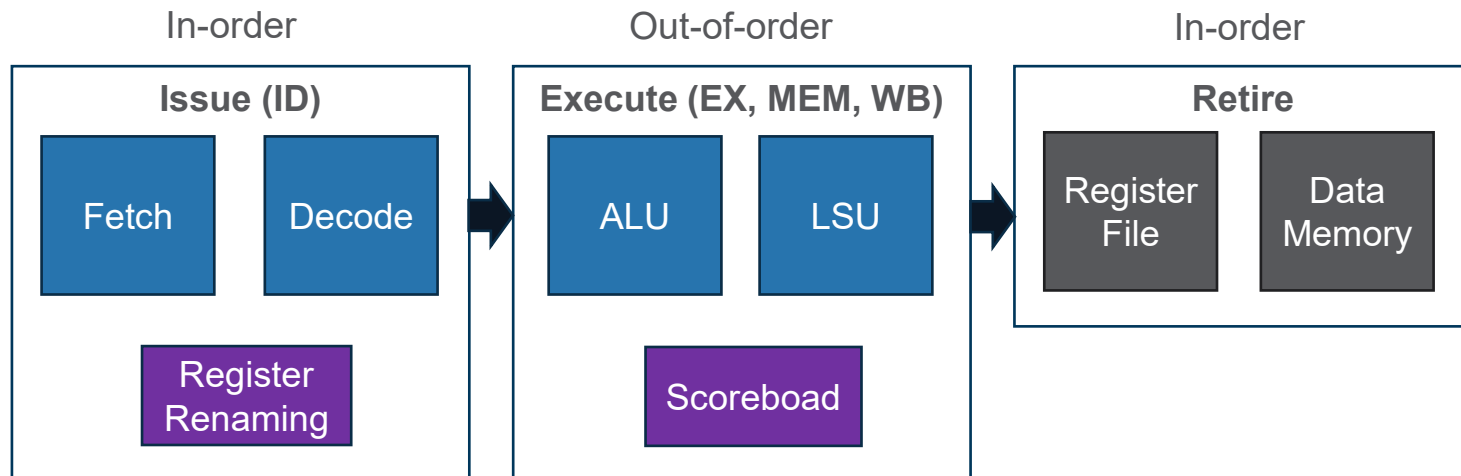


Register Renaming Summary

- Advantages?
 - Eliminate WAR data dependencies (anti-dep)
 - Eliminate WAW data dependencies (output-dep)
- Limitations
 - Doesn't remove RAW data dependencies (true-dep)

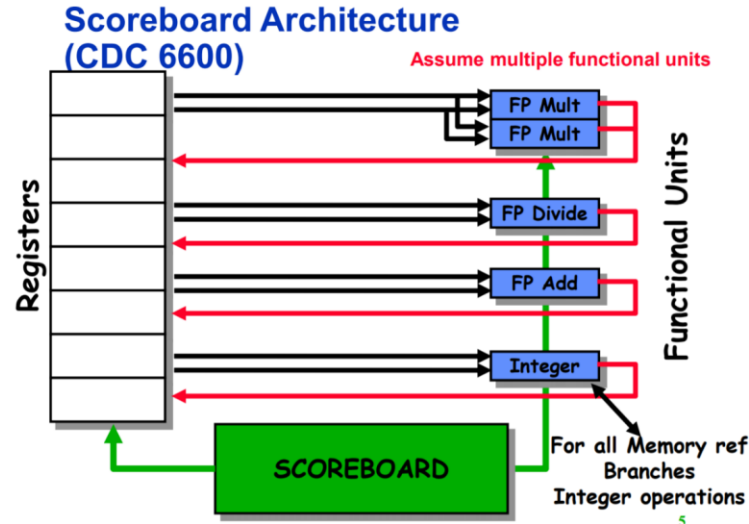
Scoreboarding

- Resolve RAW dependencies
- Enable out-of-order execution



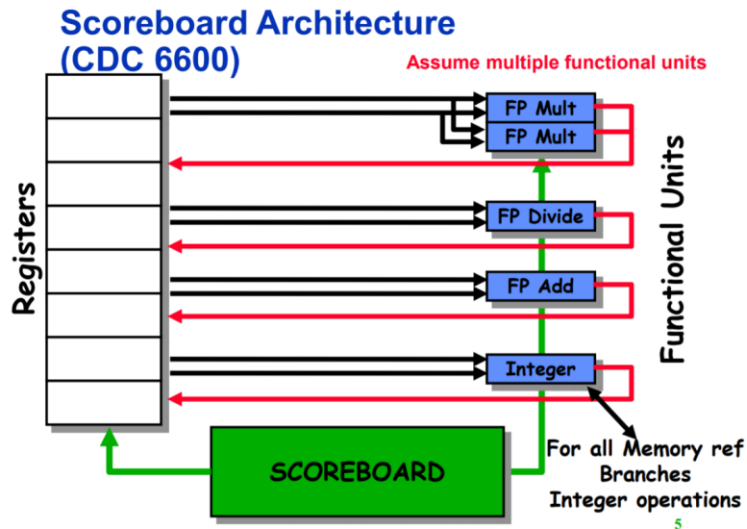
Scoreboarding

- CDC6600's scoreboard: first OOO processor!
 - 1964 - Seymour Cray from CDC



CDC6600 - Scoreboarding

- Supporting multiple fixed-latency functional units
 - FP Mult x2
 - FP Div
 - FP Add
 - Integer
 - ALU
 - Branch
 - Mem



CDC6600 – Four-Stages Pipeline

- Issue
- Register Operands
- Execute
- Writeback

CDC6600 – Four-Stages Pipeline

- Issue
 - Fetch next instruction
 - Wait/Stall for hazards
 - **Data hazards:**
 - WAR: check if previous instruction is writing to any of my source operands
 - WAW: check if previous instruction is writing to my destination operand
 - **Structural hazards:**
 - Check if functional unit not available

CDC6600 – Four-Stages Pipeline

- Read Operands
 - Read register file
 - No check is needed

CDC6600 – Four-Stages Pipeline

- Execute
 - Each functional unit execute its issued instruction
 - RAW data dependencies:
 - Wait until all source operands are available

CDC6600 – Four-Stages Pipeline

- Writeback
 - Update register file
 - Ensure program ordering
 - Wait until all prior instructions have completed before serving current one using Instruction Status state

CDC6600 – Scoreboard States

- Instruction Status: which pipeline stage it is in
- Functional Units (FU) status
 - Busy - is busy
 - Op - operation to execute
 - Fi - destination register id
 - Fj, Fk - source register ids,
 - Qj, Qk - FUs producing source registers
 - Rj, Rk - FUs producing source registers are ready
- Register result status: which FU is producing the register

CDC6600 – Scoreboard Example

- LD F6 34 R2
- LD F2 45 R3
- FMULT F0 F2 F4
- FADD F8 F6 F2
- FDIV F10 F0 F6

IS	RO	EX	WB

Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk

INT
FM1
FM2
FA
FD

F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10

CDC6600 – Scoreboard Example

- Cycle1

- LD F6 34 R2
- LD F2 45 R3
- FMULT F0 F2 F4
- FADD F8 F6 F2
- FDIV F10 F0 F6

IS	RO	EX	WB
Y			

Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Y	LD	F6		R2				Y

INT
FM1
FM2
FA
FD

F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10
						INT				

CDC6600 – Scoreboard Example

- Cycle2

- LD F6 34 R2
- LD F2 45 R3
- FMULT F0 F2 F4
- FADD F8 F6 F2
- FDIV F10 F0 F6

IS	RO	EX	WB
Y	Y		

Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk	
Y	LD	F6		R2				Y	INT
									FM1
									FM2
									FA
									FD

F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10
						INT				

CDC6600 – Scoreboard Example

- Cycle3

- LD F6 34 R2
- LD F2 45 R3
- FMULT F0 F2 F4
- FADD F8 F6 F2
- FDIV F10 F0 F6

IS	RO	EX	WB
Y	Y	Y	

Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk	
Y	LD	F6		R2				Y	INT
									FM1
									FM2
									FA
									FD

F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10
						INT				

CDC6600 – Scoreboard Example

- Cycle4

- ~~LD F6 34 R2~~
- LD F2 45 R3
- FMULT F0 F2 F4
- FADD F8 F6 F2
- FDIV F10 F0 F6

IS	RO	EX	WB
Y	Y	Y	Y

Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk	
									INT
									FM1
									FM2
									FA
									FD

F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10

CDC6600 – Scoreboard Example

- Cycle5

- ~~LD F6 34 R2~~
- LD F2 45 R3
- FMULT F0 F2 F4
- FADD F8 F6 F2
- FDIV F10 F0 F6

IS	RO	EX	WB
Y			

Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk	
Y	LD	F2		R3				Y	INT
									FM1
									FM2
									FA
									FD

F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10
		INT								

CDC6600 – Scoreboard Example

- Cycle6

- LD F6 34 R2
- LD F2 45 R3
- FMUL F0 F2 F4
- FADD F8 F6 F2
- FDIV F10 F0 F6

IS	RO	EX	WB
Y	Y		
Y			

Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk	
Y	LD	F2		R3				Y	INT
Y	FMUL	F0	F2	F4	INT		N	Y	FM1
									FM2
									FA
									FD

F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10
FM1		INT								

CDC6600 – Scoreboard Example

- Cycle7

- ~~LD F6 34 R2~~
- LD F2 45 R3
- FMUL F0 F2 F4
- FADD F8 F6 F2
- FDIV F10 F0 F6

IS	RO	EX	WB
Y	Y	Y	
Y			
Y			

Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk	
Y	LD	F2		R3				Y	INT
Y	FMUL	F0	F2	F4	INT		N	Y	FM1
									FM2
Y	FADD	F8	F6	F2		INT	Y	N	FA
									FD

F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10
FM1		INT						FA		

CDC6600 – Scoreboard Example

- Cycle8

- ~~LD F6 34 R2~~
- ~~LD F2 45 R3~~
- FMUL F0 F2 F4
- FADD F8 F6 F2
- FDIV F10 F0 F6

IS	RO	EX	WB
Y	Y	Y	Y
Y			
Y			
Y			

Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk	
Y	LD	F2		R3				Y	INT
Y	FMUL	F0	F2	F4	INT		Y	Y	FM1
									FM2
Y	FADD	F8	F6	F2		INT	Y	Y	FA
Y	FDIV	F10	F0	F6	FM1		N	Y	FD

F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10
FM1		INT						FA		FD

CDC6600 – Limitations

- No register renaming
- Detect WAR and WAW but stalls

Q&A

Acknowledgement

- This course is partly inspired and use materials from the following courses created by my colleagues:
 - ECE-M1126C: Nader Sehatbakhsh @ UCLA
 - CS3220: Hyesoon Kim @ gatech